CS320 Project Two
Jessica Soler

Summary

My testing approach for Project One began with an analysis of the requirements for the Contact, Task, and Appointment services. I separated each requirement into specific test conditions and evaluated valid inputs, invalid inputs, and boundary values. This follows the principle that requirements form part of the test basis used to determine what should be tested (Hambling et al., 2019). To evaluate the effectiveness of the final test suite, I used boundary cases, negative tests, exception tests, state verification, and code coverage as complementary forms of evidence.

For Contact, I tested the limits for the ID, first name, last name, phone number, and address. The phone number requirement was tested with null and empty values, too few and too many digits, alphabetic characters, and a valid 10-digit phone number. The service tests also addressed duplicate IDs, deletion, valid updates, and attempts to update contacts that did not exist. Task also followed the same requirement-based approach for its 10-character ID, 20-character name, and 50-character description. Appointment added a time dependent requirement, so I tested null, past, and future dates in addition to the ID and description constraints.

Boundary-value analysis was useful because defects commonly occur where valid and invalid input ranges meet (Hambling et al., 2019). For example, in TaskTest.java, lines 148-156 generate a task name one character beyond the 20-character maximum with “A”.repeat(MAX_NAME_LENGTH + 1) and use assertThrows() to verify that it is rejected. Lines 193-202 create a name that is exactly 20 characters and uses assertEquals() to confirm that the valid boundary is accepted.

Negative and exception testing allowed me to deliberately challenge the software instead of testing only the expected path. For example, ContactServiceTest.java, lines 228-251, attempts to update a contact with both a null first name and an empty first name. Other negative cases included duplicate IDs, nonexistent IDs, invalid phone numbers, too long fields, and past appointment dates. In AppointmentTest.java, lines 154-170, assertThrows() specifically verifies that null and past dates produce an IllegalArgumentException. These tests helped determine whether the software could reject invalid conditions instead of only demonstrating that valid operations worked.

State verification was also important because successful method execution does not prove that the correct result was produced. In ContactServiceTest.java, lines 199-217, two contacts are added before contactID 2 is updated. The test uses assertAll() with two assertEquals() statements to verify that Alice remains unchanged while Bob receives the new value. This confirms that the service not only executes the update, but also locates and modifies the correct object.

Finally, I used code coverage as a quantitative measure of how completely the production code was exercised. Project One finished with 64 Contact tests, 38 Task tests, and 36 Appointment tests. With a total of 138 passing tests, all six production classes reached 100% code coverage. However, I do not consider 100% coverage by itself proof that the software is correct. Testing can reveal defects and increase confidence in software quality, but it cannot prove the complete absence of defects (Hambling et al., 2019). Coverage showed that the production code had been exercised, while boundary cases, negative tests, exception tests, and state verification provide evidence that the final test suite was effective.

Writing the JUnit tests became more systematic as I moved through the three services. I learned that a useful test needed to do more than confirm that a method executed. The tests needed to verify expected behavior, expected failures, and changes in object state.

I ensured that my tests were technically sound by checking that validation rules were enforced consistently at different points in an object's lifecycle. For example, in TaskTest.java, lines 130-156 test invalid task names during construction, while lines 159-190 test the same null, empty, and over-length conditions through setName(). In both cases, assertThrows() expects an IllegalArgumentException. Testing both construction and later updates helped confirm that invalid data could not bypass validation depending on how it entered the object. I also tested Java specific behavior in AppointmentTest.java in lines 209-240 by modifying both the original Date passed to the constructor and a Date returned by the getter. The tests both use assertNotEquals() to verify that Appointment protects its internal date through defensive copying.

I made the test code more efficient and maintainable by reducing repeated setup and organizing related tests. In TaskTest.java, lines 20-58, constants and helper methods such as createValidTask() and createTaskWithName() allow test data to be reused instead of rebuilding complete objects for each condition. TaskServiceTest.java, lines 29-44, uses @BeforeEach to provide fresh setup for every test, while AppointmentTest.java, lines 25-42, creates past and future dates dynamically instead of relying on arbitrary dates that would eventually become outdated. I also used @Nested, @DisplayName, and @Tag to group and label related tests. These choices reduced duplication and made the test suite easier to read, troubleshoot, and maintain.

Reflection

I used unit testing, requirement based testing, boundary value analysis, positive and negative testing, exception testing, and state verification throughout Project One. Unit testing allowed me to evaluate individual classes and service methods independently. Requirement based testing kept the tests connected to the stated constraints. Boundary testing focused on exact limits and values immediately beyond them. Negative and exception testing challenged the software with invalid conditions. State verification checked the results of an operation instead of only confirming that the method completed. These techniques are useful in projects with clearly defined business rules because they provide fast feedback about specific behaviors and failures.

Other testing techniques were outside the scope of this project. I did not use integration testing, end-to-end testing, performance testing, or security testing. Integration testing would become important if these services communicated with a database, API, or other components. End to end testing would evaluate complete user workflows. Performance testing would be appropriate for systems expected to handle high volumes of users or data. Security testing would be necessary for applications containing authentication or sensitive information. The appropriate technique depends on the risks, architecture, and purpose of the software being developed.

My mindset became more cautious as the project progressed. Instead of only asking whether the code worked under normal conditions, I began asking how it could fail. Testing null values, boundaries, duplicate IDs, nonexistent IDs, and incorrect dates required me to consider conditions that a successful example would not reveal. This was important because the classes were interconnected. A service might locate the correct object but still depend on that object's setter to reject an invalid update. Additionally, the Appointment class required additional consideration because Java's mutable Date objects could expose internal state.

Bias was also a concern because I wrote both the production code and the tests. It would have been easy to design tests around what I expected the implementation to do instead of what the requirements. I tried to limit that bias by returning to the written requirements and utilizing instructor feedback to revisit areas that needed stronger validation. That helped shift my focus from proving that my code worked to looking for conditions that could make it fail.

Discipline is important because cutting corners in development or testing can create technical debt that becomes more expensive to correct later. In this project, revisiting earlier tests, strengthening coverage, and returning the full suites required additional work but produced a more reliable test suite. As a software engineering professional, I plan to reduce technical debt by writing tests alongside features, maintaining regression tests when defects are discovered, refactoring duplicated code, reviewing coverage without treating it as proof of correctness, and using peer review to challenge assumptions I may overlook in my own work.

References

Hambling, B., Morgan, P., Samaroo, A., Thompson, G., & Williams, P. (2019). *Software testing: An ISTQB-BCS certified tester foundation guide* (4th ed.). BCS Learning & Development Limited.